ATC'26 · HONG KONG · NOVEMBER 2026

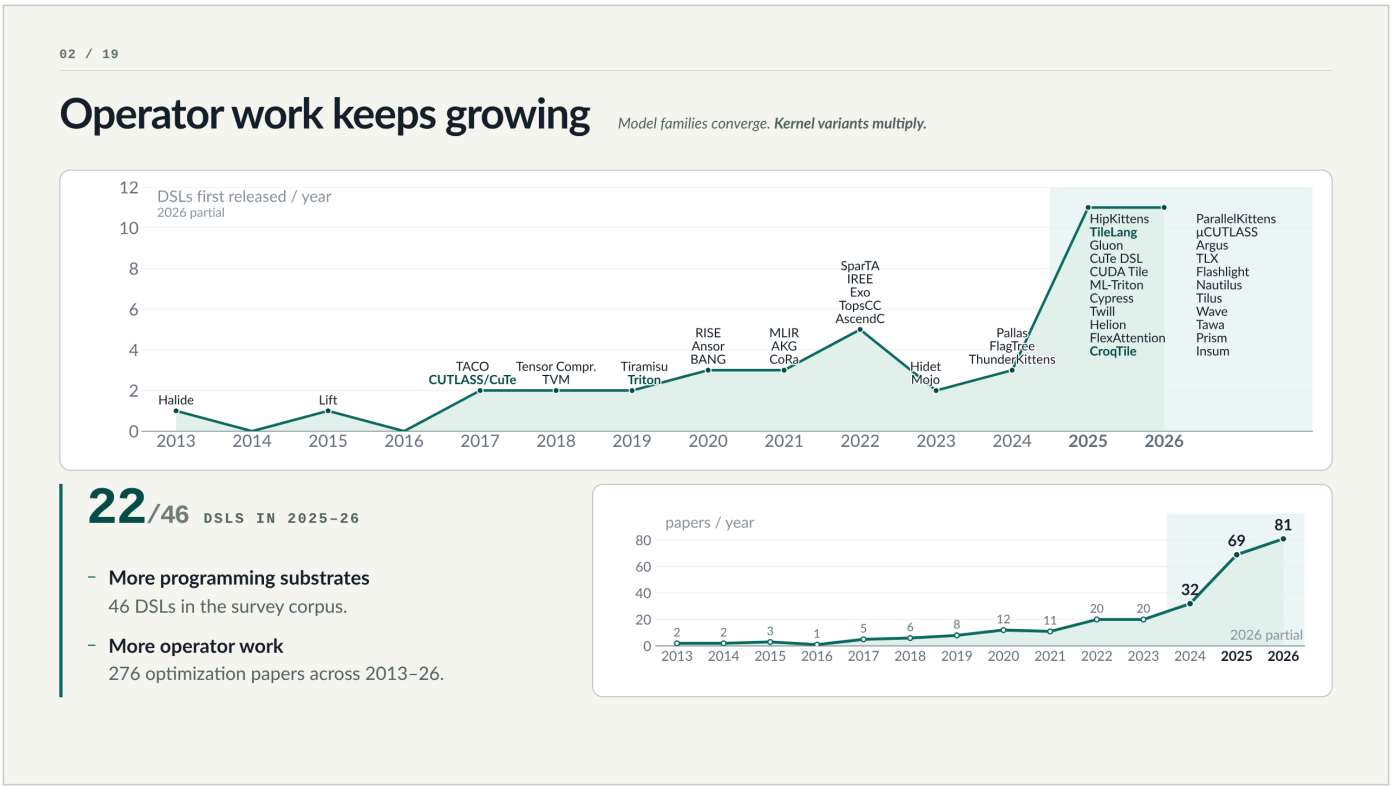
A Novel AI-Native GPU Kernel Programming Language for Autonomous Kernel Tuning

Heng Shi · Xiaofeng Guan · Sijie Wu · Enming Fan · Chenpeng Wu
Qiqi Gu · Jiahang Xie · Guoqing Bao · Jianguo Yao

Shanghai Enflame Intelligence & Shanghai Jiao Tong University

P1 · Title · 0:00–0:20

Hi everyone, I am Heng Shi, from Shanghai Enflame Intelligence and Shanghai Jiao Tong University. I am happy to be here to present our work on CroqTile, a new GPU programming language designed for autonomous kernel tuning. Let us start with the motivation.

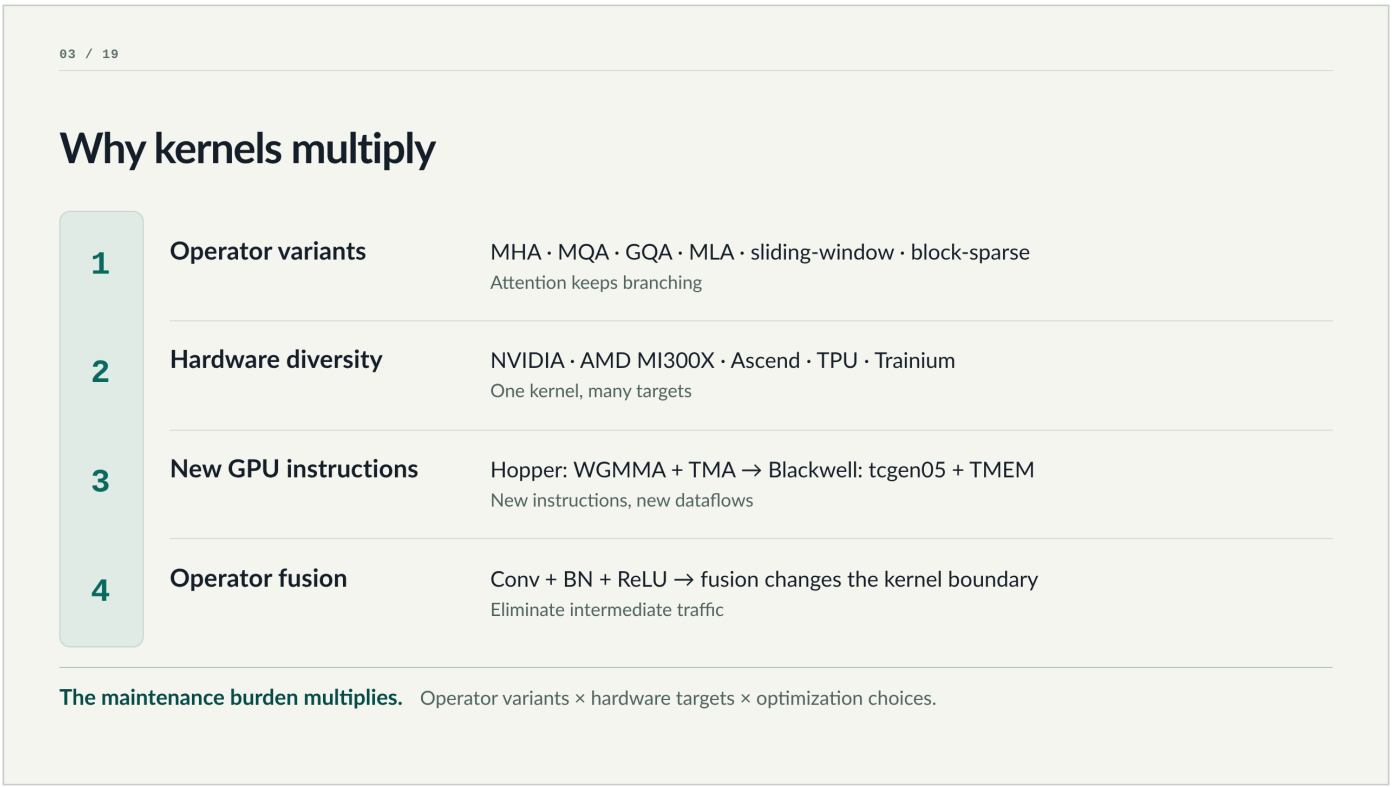


P2 · Operator work keeps growing · 0:20–0:55

So, model architectures are gradually converging into a few families. But if you work on the kernels underneath, there is more and more work to do, right?

As the top figure shows, new GPU DSLs keep coming out. In our survey, 22 out of 46 appeared in 2025 and 2026. The lower figure shows the same trend for operator optimization papers.

So why does the kernel work keep growing, even when the models look similar?



P3 · Why kernels multiply · 0:55–1:50

We think there are four reasons, and they stack together.

First, the operators themselves keep changing. Even for attention, we have MHA, MQA, GQA, MLA, and more variants. The model is still a transformer, but the kernel is already different.

Second, we have more hardware platforms. A good kernel for NVIDIA does not automatically become a good kernel for AMD or Ascend.

And even within NVIDIA, the instructions keep changing. We just figured out how to use Hopper's warpgroup MMA, and now Blackwell brings another instruction family and another memory to manage.

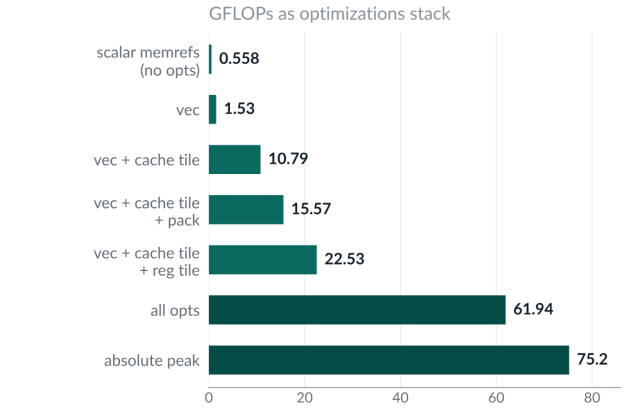
Finally, we fuse operators together to avoid intermediate memory traffic. This also creates new kernels.

Together, these make a fixed library of standard operators harder to maintain.

Kernel optimizations interact

COUPLED PERFORMANCE DECISIONS

- **Locality**
  Tiling · data reuse · L2 bypass / cache policy
- **Data movement**
  Swizzling · vectorization · async copies
- **Execution overlap**
  Multibuffering · warp specialization · scheduling
- **Compute & cooperation**
  Unrolling · approximation · sparsity · fusion
  MMA layouts · low-precision MMA · clusters / DSMEM



A larger tile improves reuse... ...but consumes more shared memory, limiting stages and overlap.

P4 · Kernel optimizations interact · 1:50–2:30

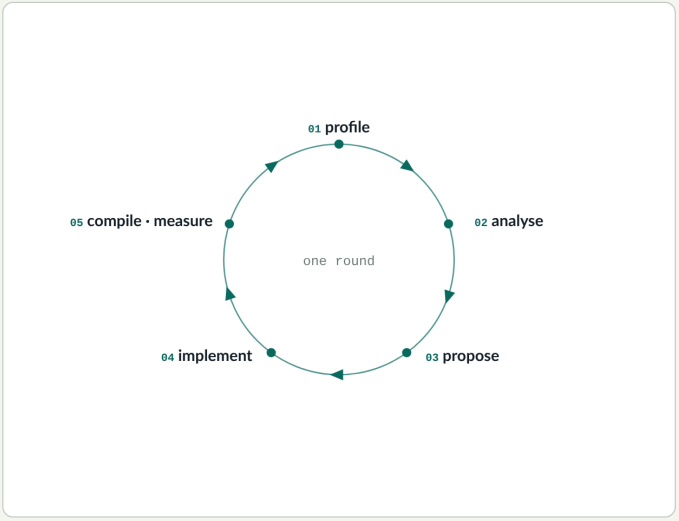
And for each kernel, there are already so many things to consider. Tiling, data reuse, swizzling, multibuffering, warp specialization, and all these tricks on the left.

Like the right figure shows, applying optimizations can move performance by orders of magnitude. So we really have to do them.

But! They are coupled. A larger tile may improve reuse, but it also takes more shared memory. Then fewer pipeline stages fit, and the overlap changes again.

So the difficulty is figuring out which combination works together. Now, how do we normally do this?

Bottleneck-driven tuning



1. Profile
2. Identify bottleneck
3. Choose structural edit
4. Implement
5. Compile · measure · keep / reject

- **Example: tensor cores waiting for data**
  Introduce a producer–consumer pipeline.
- **Expert effort in every round**
  Interpret counters, redesign the kernel, validate the edit.

P5 · Bottleneck-driven tuning · 2:30–3:10

The first way is probably familiar to you.

We run the kernel, profile it, and figure out the bottleneck. Then we choose a structural change, implement it, compile, and measure again. If it helps, we keep it; otherwise, we try something else.

For example, if the tensor cores are waiting for data, we may introduce a producer-consumer pipeline. That can change quite a lot of code.

This is powerful, cause the expert can redesign the kernel. But every round needs someone to interpret the feedback and decide the next change.

Template search



`Tile size · stages · warps · swizzle`

- **Fixed skeleton**
  Choose the implementation structure up front.
- **Parameter search**
  Enumerate tile sizes, stages, warps and swizzles.
- **Filter, then benchmark**
  Reject resource violations; measure valid candidates.

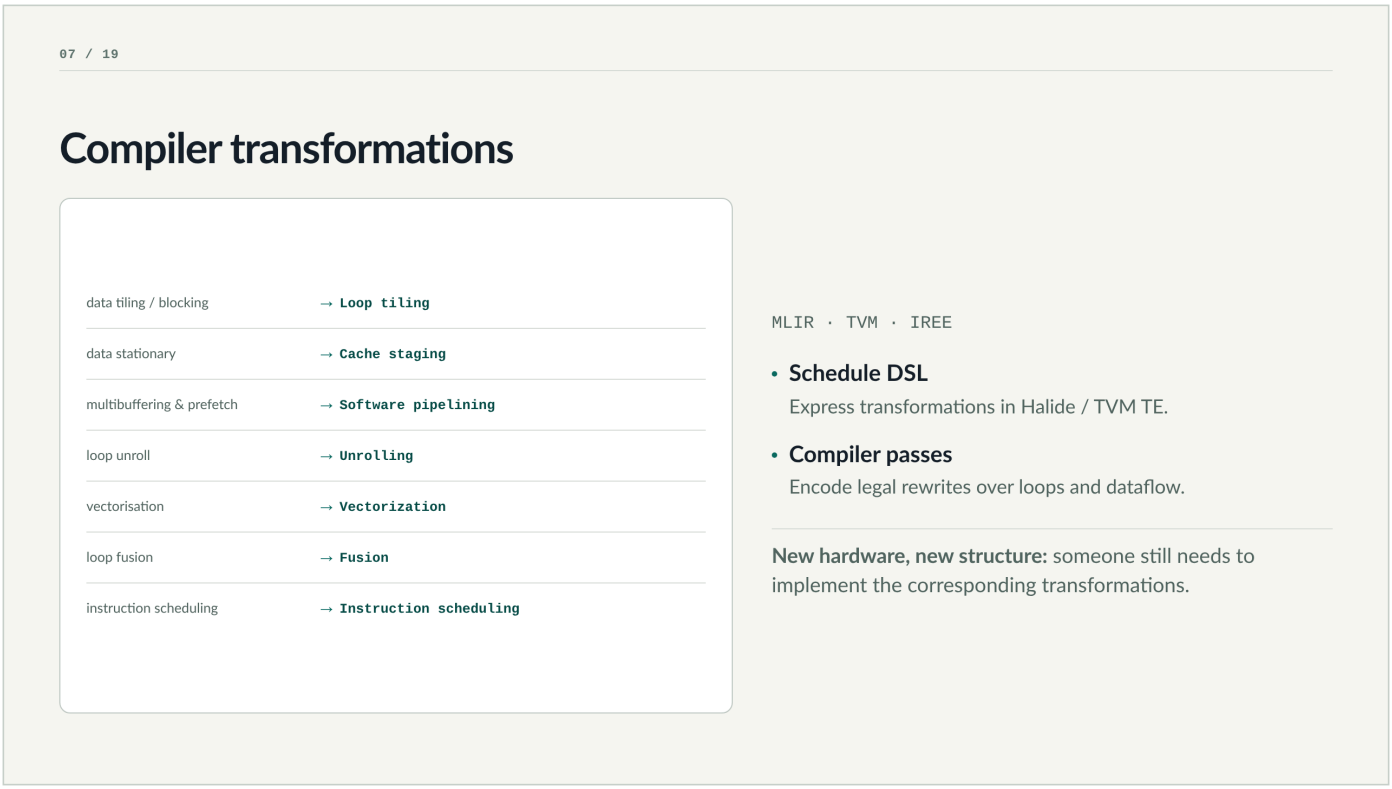
Boundary: a new producer–consumer structure must first be exposed in the template.

P6 · Template search · 3:10–3:45

The second way is autotuning. We write a template with tunable parameters, like tile size, stages, warp count, and swizzle.

As the figure shows, each point is one configuration. We reject the ones that exceed resource limits, then benchmark the remaining candidates.

But the search stays inside the template. If the better kernel needs a different producer-consumer structure, and we did not expose that choice, the search cannot discover it.



P7 · Compiler transformations · 3:45–4:15

The third way is to let the compiler do the transformations.

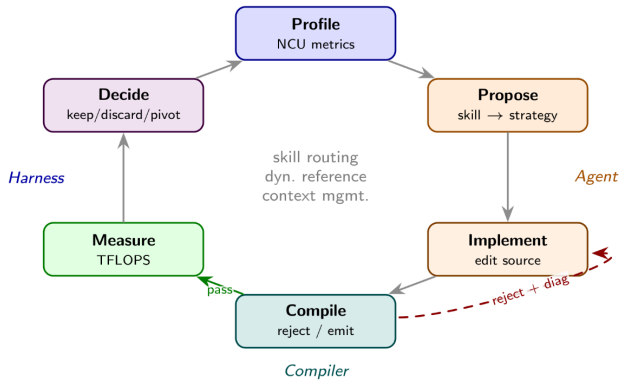
You can see the correspondence here. Tiling becomes a loop transformation, data reuse becomes cache staging, and multibuffering becomes software pipelining.

This is how systems like TVM and MLIR help us. But when a new hardware feature requires a new kernel structure, somebody still has to implement the corresponding transformations.

Agents rewrite the kernel

PROFILE → EDIT → EVALUATE

Full source edits · structural + parametric



WORKFLOW	Planning · search · refinement
EVALS	Correctness · speed · generalization
TOOLING	Syntax–context · compiler–harness co-design
KNOWLEDGE	Skills · references · history

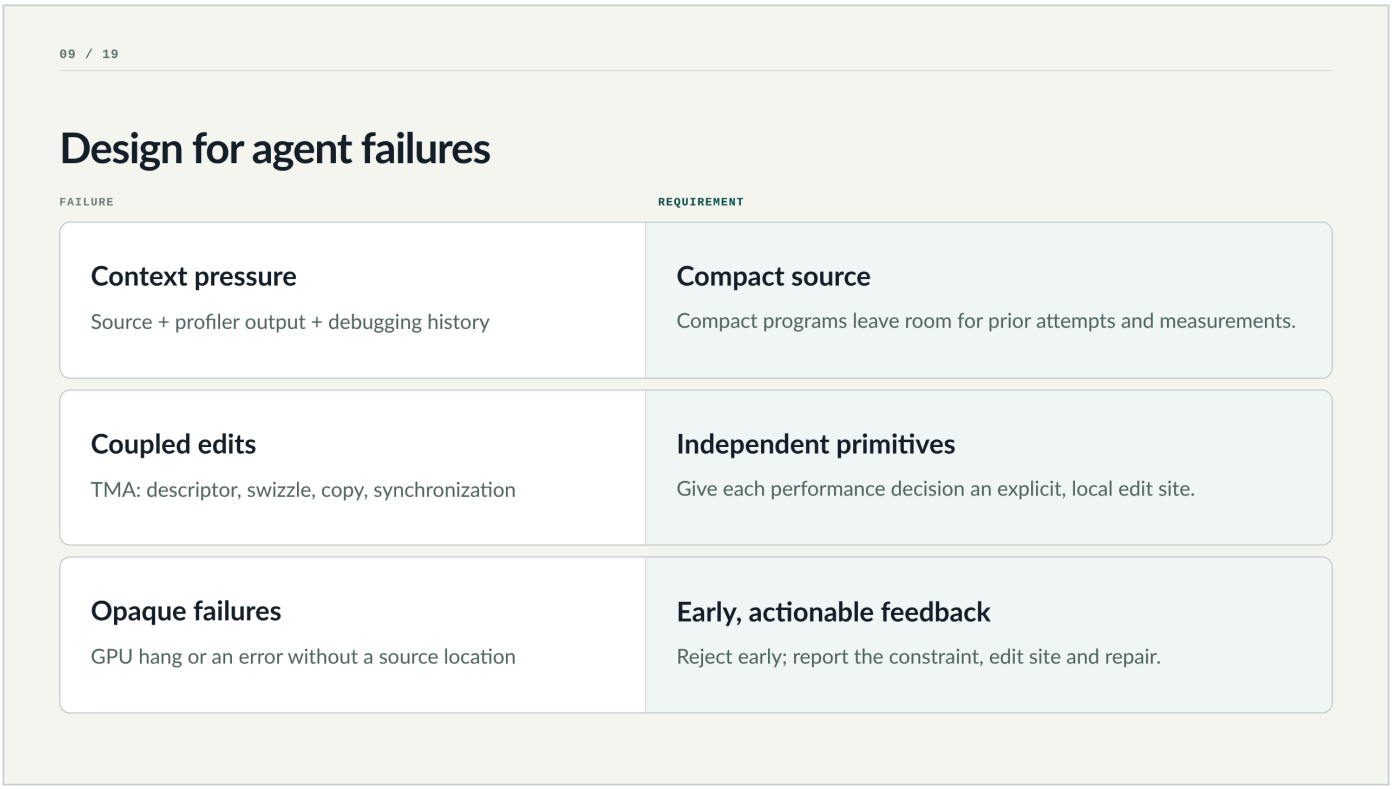
P8 · Agents rewrite the kernel · 4:15–4:55

OK, now what we gonna do? Recently, people have started putting coding agents into this loop.

The agent can read the source and profiler feedback, propose a change, and rewrite the kernel. So it can change both the parameters and the structure, much like the expert loop we just saw.

There are many efforts on planning, evaluation, tools, and knowledge, as listed on the right.

For this work, we focus on one question: what kind of programming language and compiler feedback would make this agent more effective? This is where CroqTile comes in.



P9 · Design for agent failures · 4:55–5:50

So first, let us see where the agent gets into trouble.

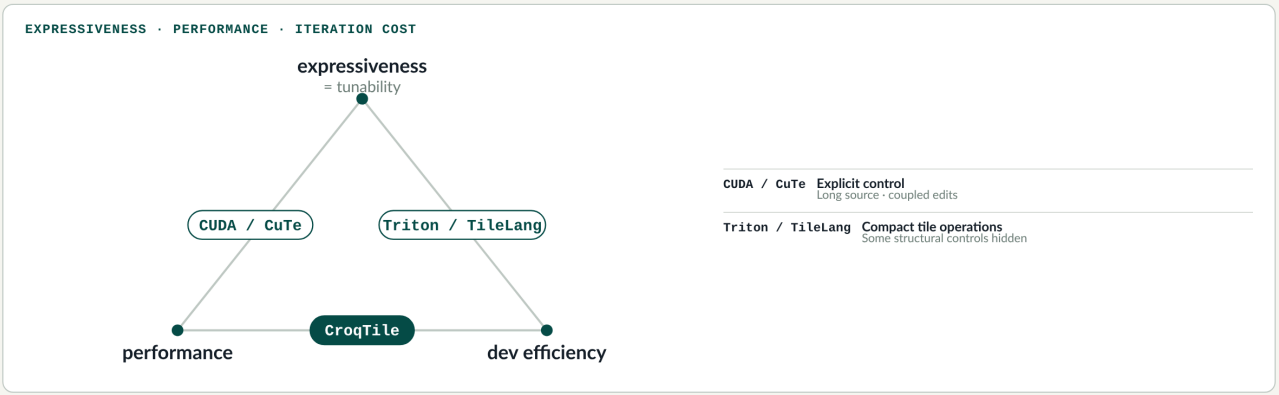
One is context pressure. A production kernel can take hundreds of lines, and every round adds source code, profiler output, and debugging messages. Eventually, the earlier attempts get pushed out, even though the agent needs them to understand what it already tried.

Second, one change can touch many places. Enabling TMA may involve the descriptor, the swizzle, the copy, and synchronization. Miss one of them, and you may get an annoying runtime bug.

Third, the feedback itself can be hard to use. A GPU hang tells you very little about which line to fix.

These give us three requirements: compact source, independent primitives, and early feedback that tells the agent how to repair the program.

CroqTile: two co-design axes



- **Syntax–context co-design**
  Compact syntax and fewer coupled edit sites leave room for source, feedback and tuning history.
- **Compiler–harness co-design**
  Symbolic checks and structured diagnostics guide rejection, repair and GPU evaluation.

P10 · CroqTile: two co-design axes · 5:50–6:30

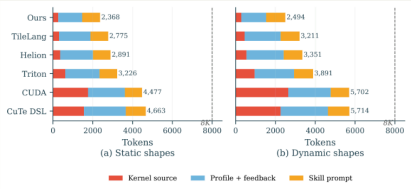
But we cannot simply make everything high-level, cause we still want the performance controls.

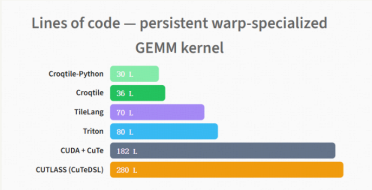
CUDA and CuTe let us express low-level decisions, but the source can be long and the edits are coupled. Tile languages make common operations much simpler, while some structural choices are hidden behind the abstraction.

We organize the design along two axes: syntax–context co-design and compiler–harness co-design.

The first makes source and structural edits compact enough to preserve tuning history. The second connects compiler checks and diagnostics to the harness, so it can reject, repair, and evaluate candidates.

Compact syntax preserves context







- **36 vs. 182 source lines**
  Same persistent, warp-specialized GEMM; CroqTile vs. CUDA + CuTe.
- **Room for tuning history**
  ≈2.37K vs. ≈4.48K tokens per static-shape attempt, including source, feedback and skills.

P11 · Compact syntax preserves context · 6:30–7:25

Let us see what this looks like. I am not gonna go through the whole grammar on the right, but the lower-left figure gives a concrete example.

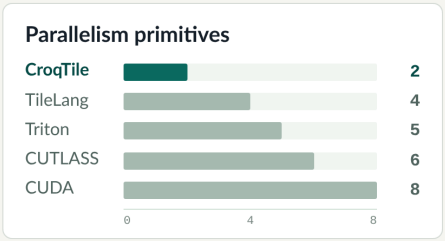
For the same persistent, warp-specialized GEMM, CroqTile takes 36 lines, compared with 182 for CUDA with CuTe.

And the upper figure asks what this means inside an agent's context. Counting source, feedback, and skills, one static-shape attempt takes about 2,370 tokens, compared with about 4,480 for CUDA.

Using the same eight-K working set for comparison, this leaves 1.6 times as much room for history. Eight-K here is just a comparison budget.

So the saved space can hold previous attempts and measurements. That is useful information for deciding the next edit.

parallel: one construct, three levels



- **One binding, three levels**
  `block` · `group-4` · `thread`
- **Explicit producer–consumer roles**
  `inthreads.async` separates TMA producers and MMA consumers.
- **Fewer coupled edit sites**
  Change the parallel binding without reconstructing thread-index calculations.

CroqTile | TileLang | CuTe / CUTLASS | CUDA

```
parallel {bm, bn}
  by [cdiv(M, WARP_M), cdiv(N, WARP_N)] : block {
  parallel p by 1 : group-4 {
    mma.row.row mc, ma, mb;
  }
  parallel {m, n}
    by [WARP_M, 2] : thread { ... }
}

// Producer–consumer warp specialization
parallel p1 by 2 : group-4 {
  inthreads.async (p1 == 0) { /* TMA producer */ }
  inthreads.async (p1 == 1) { /* MMA consumer */ }
}
```

P12 · parallel: one construct, three levels · 7:25–8:15

Now let us see two syntax examples. The first one is parallelism.

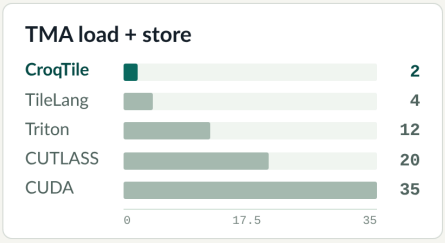
[Point to the three `parallel` blocks on the right.]

You can see the same construct at three levels: blocks, warpgroups, and threads. We specify how the work is divided and which level executes it.

Then look at the bottom example. We create two warpgroups and give them different jobs. One does the TMA data movement, and the other does the MMA computation. This is the producer-consumer structure we mentioned earlier.

The useful part for tuning is that this decision has an explicit place in the source. The agent can change the parallel structure without reconstructing all the thread-index calculations by hand.

tma: one statement per transfer



- **One statement per transfer**
  Source and destination specify the tile movement.
- **Swizzle as a modifier**
  `.swiz<128>` makes the layout choice explicit at the transfer.
- **Compiler-generated descriptor**
  The agent edits the transfer; the compiler keeps the derived descriptor consistent.

CroqTile | Triton | TileLang | CuTe / CUTLASS | CUDA

```
// LOAD · global → shared
tma.copy.swiz<128>
  lhs.subspan(WARP_M, TILE_K)
     .at(bm, iv_k) => lhs_s;

// STORE · shared → global
tma.copy out_s =>
  output.subspan(WARP_M, WARP_N)
        .at(bm, bn);
```

P13 · tma: one statement per transfer · 8:15–9:05

The second example is data movement. Remember that enabling TMA could require changing several coupled pieces?

Here, one statement describes the transfer. The source and destination tell us which tile moves where, and the swizzle is written as a modifier. The compiler derives the corresponding descriptor and transfer machinery.

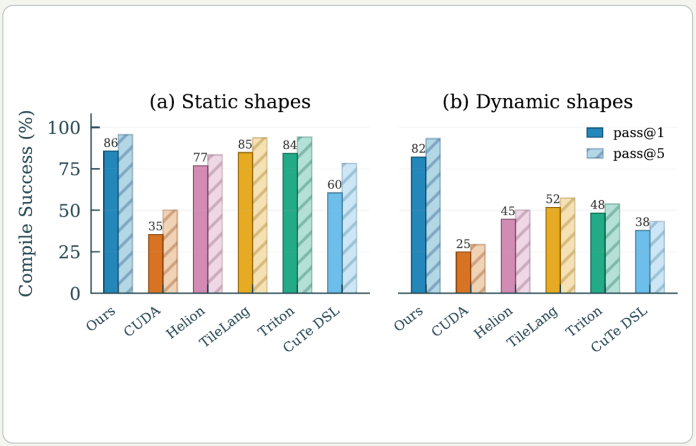
[Point to the load, then the store.]

So this example needs two statements, one for loading and one for storing. The comparison on the left shows how much supporting code the other implementations require.

We still expose the performance choice, like which swizzle to use. But the agent does not have to manually keep every derived detail consistent with that choice.

Compile success across shapes

PASS@1 / PASS@5 · STATIC + DYNAMIC



pass@1: 85.7% static · 82.1% dynamic
Dynamic shapes: Triton 48% in this evaluation.

1 Less coupled syntax

Each optimization touches fewer interdependent code sites.
Fewer details to keep consistent → fewer edit errors.

2 Symbolic inference

The type system and compiler infer shape relationships and check constraints.
Less manual shape bookkeeping, especially for dynamic shapes.

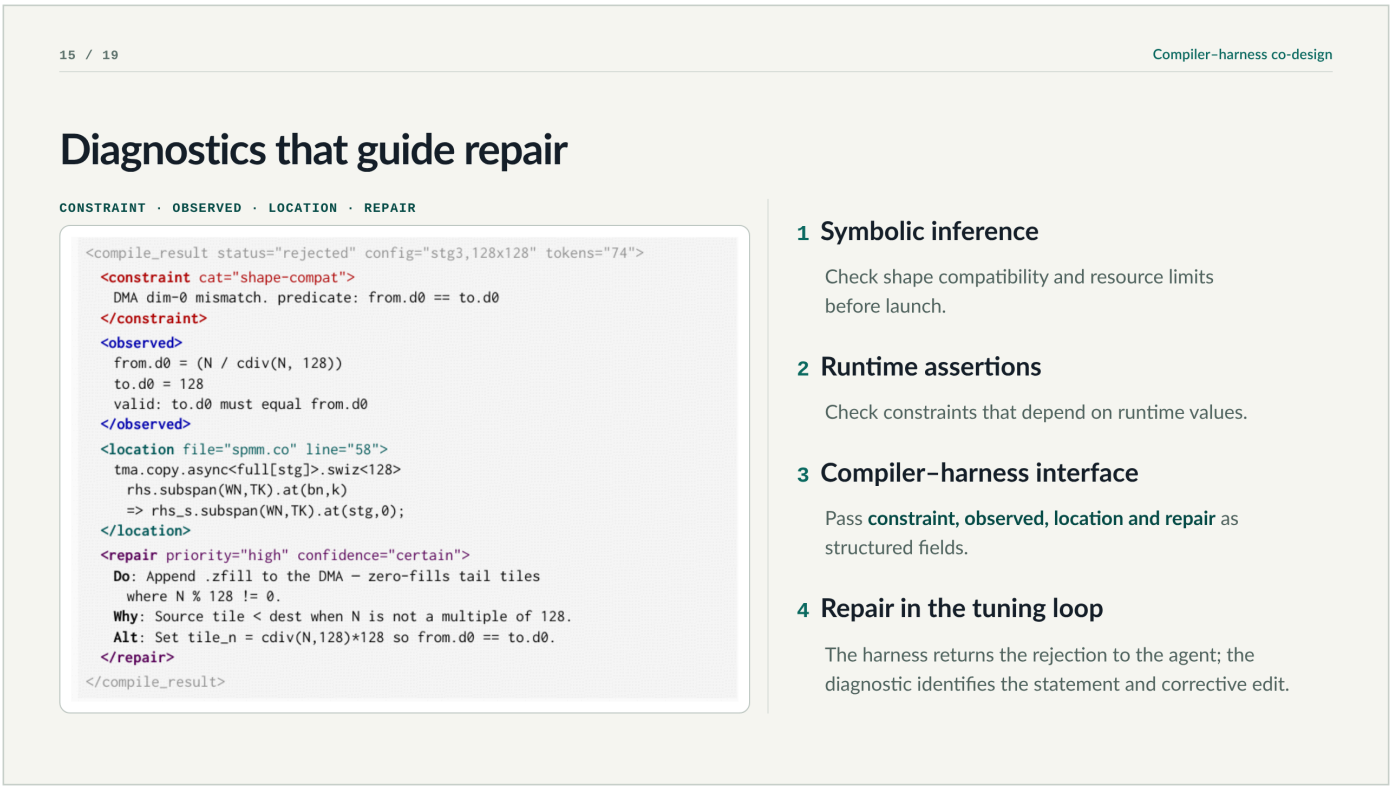
P14 · Compile success across shapes · 9:05–9:45

So does this actually help the agent write a valid program?

Here we compare compile success under the same agent, hardware, and budget. For static shapes, CroqTile reaches 85.7 percent on the first attempt. For dynamic shapes, it stays at 82.1 percent, while Triton reaches 48 percent in this evaluation.

There are two mechanisms here. First, the syntax has fewer coupled edit sites, so the agent has fewer details to keep consistent. Second, the type system and compiler use symbolic inference to derive shape relationships, which matters especially for dynamic shapes.

Correctness still needs verification. But fewer compile repairs leave more budget for performance tuning.



P15 · Diagnostics that guide repair · 9:45–10:40

And what happens when the program does fail? This is where compiler–harness co-design comes in.

The symbolic engine checks shapes and resources before launch. For constraints that depend on runtime values, we also have runtime assertions.

Then the compiler and harness return structured feedback. Let us follow the example on the left.

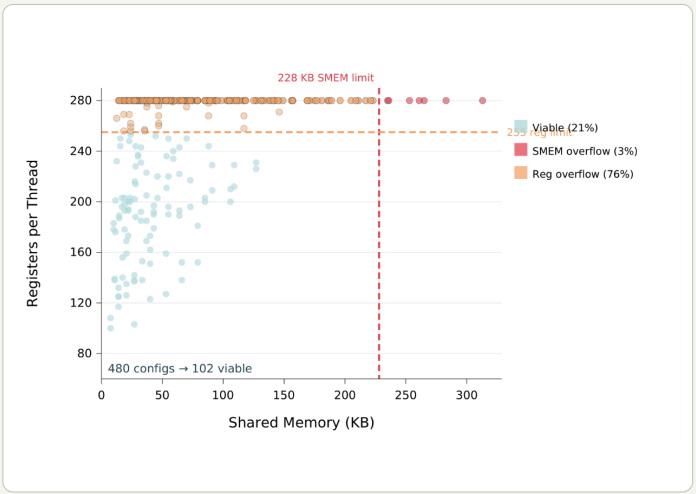
[Follow the four colored blocks from top to bottom.]

First, which constraint failed? The source and destination dimensions do not match. Second, what values did we observe? Third, which statement caused it? Finally, how can we repair it?

In this case, the message suggests adding zero fill for the tail tile. So the next edit is concrete. The agent can go directly to that statement, apply the repair, and compile again.

Prune before GPU execution

SPARSE GEMM · CONFIGURATION PRUNING



- **353 checks across 7 modules**
  Compiler checks cover shapes, resources and synchronization.
- **480 configurations → 102 viable**
  79% are pruned before GPU execution, mostly for register and shared-memory limits.
- **The harness measures viable candidates**
  Rejected candidates return with a repair hint; passing candidates proceed to correctness checks and timing.

P16 · Prune before GPU execution · 10:40–11:30

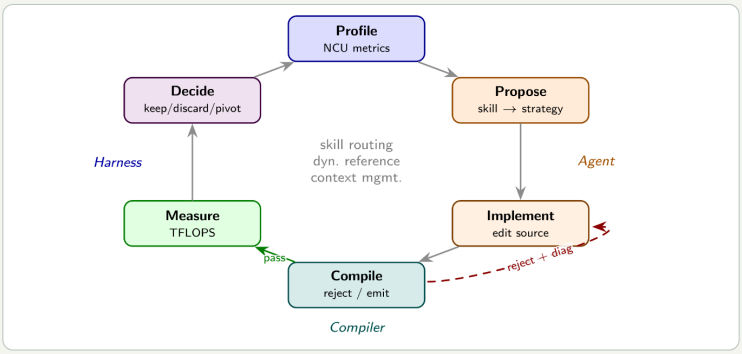
We can also use these checks to avoid wasting GPU trials.

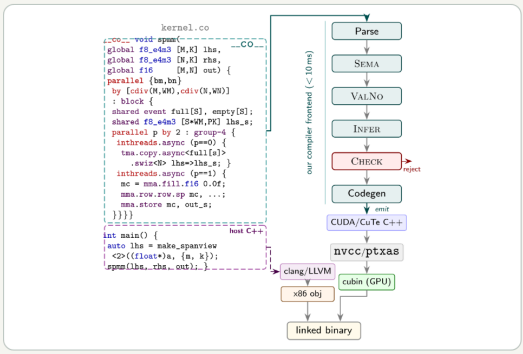
The compiler has 353 checks across seven modules. In this sparse GEMM sweep, we start with 480 configurations, and only 102 remain viable. About 79 percent are pruned before GPU execution, mostly because of register and shared-memory limits.

And the feedback quality matters for the final performance too. In the sparse GEMM ablation, replacing the structured feedback with generic errors reduces throughput by 14.9 percent.

Why would an error message affect throughput? Cause the agent has a limited tuning budget. If it spends that budget guessing how to repair invalid programs, it has fewer chances to find a faster valid one.

Close the autonomous loop





- **Syntax–context co-design**
  Compact source and local edits keep prior attempts and measurements available to the agent.
- **Compiler–harness co-design**
  Structured rejection feeds the next repair; valid candidates proceed to verification and measurement.

1.23× vendor libraries Geometric-mean speedup across evaluated workloads with vendor references.

P17 · Close the autonomous loop · 11:30–12:25

Now we can put both co-design axes together in the tuning loop.

One agent follows these six phases: profile, propose, implement, compile, measure, and decide. If compilation rejects the candidate, the feedback goes straight back to implementation. Once the candidate passes the checks, we verify correctness and measure its performance.

We also provide skills and references, so the agent has the language rules and relevant optimization ideas when it needs them.

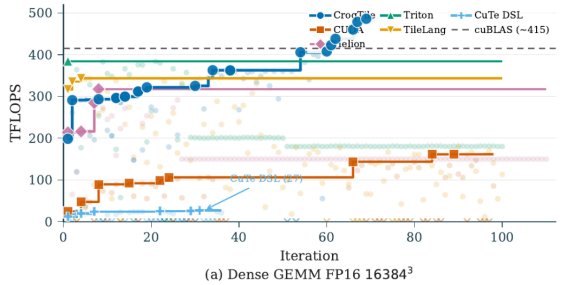
Remember the expert loop at the beginning? We are still doing that kind of iterative work. Here, the compact program and actionable feedback help the agent carry it out.

Across the evaluated workloads with vendor references, the resulting kernels achieve a geometric-mean speedup of 1.23 times over the vendor libraries. Let us look at two tuning trajectories.

Tuning beyond cuBLAS

Dense FP16 GEMM

M = N = K = 16,384

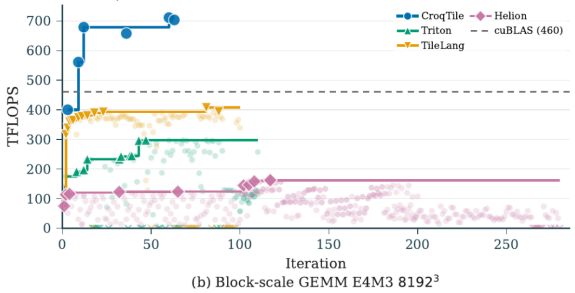


(a) Dense GEMM FP16 16384^3

Improves steadily, reaches cuBLAS, then exceeds it.

Block-scale E4M3 GEMM

M = N = K = 8,192



(b) Block-scale GEMM E4M3 8192^3

Crosses the vendor baseline early and continues improving.

- **Read the curves**
  Solid lines: running-best TFLOPS; dashed line: cuBLAS.
- **Shared evaluation protocol**
  H800 PCIe · Claude Sonnet 4.6 · same agent and per-shape time cap.

P18 · Tuning beyond cuBLAS · 12:25–13:20

On the left is dense FP16 GEMM, with all three dimensions equal to 16,384. The horizontal axis is tuning iteration, and the vertical axis is throughput. The solid lines track the best result found so far.

[Trace the blue curve, then point to the dashed line.]

You can see CroqTile gradually improves, reaches the cuBLAS line, and then goes beyond it.

On the right is block-scale E4M3 GEMM, at 8,192 cubed. Here it crosses the vendor baseline much earlier and keeps improving.

What I would like you to notice is the progress across iterations. In these two cases, the agent can keep turning its proposed changes into working, faster kernels. This is the outcome we want from the language and compiler working together.

Thank you



CroqTile is open source

Scan · CroqTile repo

ACCEPTED WORKS USING CROQTILE

- **SC'26** · Structured Sparse GEMM on Hopper via Analytical Modeling
- **SC'26** · Do We Need Tensor Cores for Stencil Computations? **Best Paper Nominee**
- **ATC'26** · An AI-Native GPU Kernel Programming Language for Autonomous Tuning
- **ATC'26** · SptcAttn: Long-Context Attention with Diagonal-Stripes Sparse Tensor Core
- **CGO'27** · Signal-Aware Liveness Analysis for Shared Memory in Warp-Specialized Kernels

P19 · Thank you · 13:20–14:00

So to wrap up, I would like you to take away two things.

First, compact source with explicit performance controls helps the agent explore kernel designs.

Second, early rejection with a concrete repair helps it use the tuning budget effectively.

CroqTile is open source, with five accepted works using it. Please feel free to scan the code and have a look.

Yeah, that is all from me. Thank you, and happy to take your questions.